

AI-Enhanced Fast Cosmological Simulations for Accurate Large-Scale Structure Inference

David Amrein

July 7, 2026

Contents

Abstract

1 Introduction

2 Theoretical Background

2.1 Cosmological Framework

2.1.1 The Cosmic Web

On scales of a few to a few hundred megaparsecs, matter in the Universe is not smoothly distributed but arranged into an interconnected network of dense compact knots, elongated filaments, two-dimensional sheets, and large under-dense voids – collectively known as the *cosmic web* [bond’how’1996, libeskind’tracing’2018]. The term originates with bond’how’1996, who showed that this filament-dominated pattern is already present in embryonic form in the tidal field of the initial density perturbations, with the nonlinear gravitational evolution sharpening rather than creating it [bond’how’1996]. The same gravitational instability that seeds this pattern is responsible for its growth: initially small density fluctuations are amplified by their own self-gravity, so that overdense regions contract while voids expand and come to dominate the volume of the Universe [bond’how’1996]. Large galaxy redshift surveys – 2dFGRS, SDSS, and the 2MASS redshift survey (2MRS) among them – as well as cosmological N -body simulations both recover this same web-like arrangement of galaxies and dark matter [libeskind’tracing’2018].

Gravitational instability drives the emergence of anisotropic structure: because collapse along the direction of a region’s strongest tidal compression proceeds fastest, an initially only mildly non-spherical overdensity first flattens into a sheet, then contracts further into a filament, and only finally collapses fully into a compact halo – the sequence of anisotropic collapse first described by Zel’dovich [libeskind’tracing’2018]. This picture can be made quantitative by looking at the local tidal (or velocity-shear) field. Following the tidal-tensor classification reviewed by [libeskind’tracing’2018], one computes the Hessian of the (rescaled) gravitational potential ϕ ,

$$T_{\alpha\beta} = \frac{\partial^2 \phi}{\partial x_\alpha \partial x_\beta}, \quad \nabla^2 \phi = \delta, \quad (1)$$

where $\delta = (\rho - \bar{\rho})/\bar{\rho}$ is the dimensionless matter overdensity and ϕ has been normalised by $4\pi G\bar{\rho}$ so that Eq. (??) takes the form of the Poisson equation [libeskind’tracing’2018]. Diagonalising the (real, symmetric) tidal tensor at each point gives three eigenvalues $\lambda_1 > \lambda_2 > \lambda_3$; counting how many of them exceed a chosen threshold λ_{th} classifies that point as part of a void (zero eigenvalues above threshold), a sheet (one), a filament (two), or a knot (three)

[libeskind

2.1.2 Matter Power Spectrum and Transfer Function

While the tidal-tensor classification above describes the cosmic web *qualitatively*, its statistical properties are quantified through the matter power spectrum $P(k)$. Writing the density contrast in Fourier space as $\delta(\mathbf{k})$, and assuming statistical homogeneity and isotropy, the power spectrum is defined through

$$\langle \delta(\mathbf{k}) \delta^*(\mathbf{k}') \rangle = (2\pi)^3 \delta_D(\mathbf{k} - \mathbf{k}') P(k), \quad (2)$$

where δ_D is the Dirac delta and the angle brackets denote an ensemble average over realisations of the initial conditions. [dodelson'modern'2020, huterer'growth'2023] Simplified we can express it

$$P(k) \propto k^{n_s} T^2(k) \quad (3)$$

with n_s the scalar spectral index. [huterer'growth'2023, eisenstein'baryonic'1998] Further we can simplify the transfer function $T(k)$ to

$$T(k) = 1 - \frac{k}{k_{eq}} + \dots \quad (4)$$

with $k_{eq} = H_0 \sqrt{\frac{2\Omega_m}{\Omega_r}}$. [bardeen'statistics'1986, eisenstein'baryonic'1998]

We see that the transfer function is an essential component of the power matter spectrum, as also the matter and radiation density. We also see that transfer function is dependent on the neutrino mass, as visualized in Figure ???. There are three red curves regarding the neutrino mass $m_{\nu_1} = 0$, $m_{\nu_2} = 0.009$ and $m_{\nu_3} = 0.05$ eV. [lesgourgues'neutrino'2012, lesgourgues'massive'2006] Also this will have a an effect on angular power spectrum C_l , which is defined as

$$C_l \approx \int k^2 P(k) dk \quad (5)$$

is the fourier transfrom of the power matter spectrum. [seljak'line'1996]. This fundamental for this project since later we will see its influence, when generating the initial conditions for the simulations.

Obtaining $P(k)$ precisely, including this transfer function, requires solving the coupled linear Einstein–Boltzmann equations for the perturbed Friedmann–Lemaître–Robertson–Walker metric and the phase-space distributions of all matter and radiation species – the same system solved by the differentiable Einstein–Boltzmann module of the Disco-DJ framework used later in this thesis [hahn'disco-dj'2024].



Figure 1: [lesgourgues’ massive’2006]

2.1.3 Lightcones and Snapshots

An N -body simulation naturally outputs *snapshots*: the full particle phase-space (positions and velocities) within a periodic box of fixed comoving side length, at a fixed time. Reproducing how we actually observe the Universe, however, requires a *lightcone*: rather than a single time slice, an observer at a fixed spacetime point receives light emitted from progressively more distant (and hence earlier) shells as one looks to higher redshift, so a simulated lightcone must record particles at the (redshift-dependent) comoving distance from which they are being observed, not at a single global time.

Because the depth of the desired lightcone (out to some z_{max}) is typically larger than a single simulation box, the box is periodically replicated around a centrally placed observer: box replication takes advantage of the fact that a simulation with periodic boundary conditions can, in principle, represent an arbitrarily large volume, at the cost of the same structures reappearing at multiple replica images along a given line of sight [kacprzak’cosmogridv1’2023]. This replication strategy, and the trade-off between box size, structure repetition, and the resulting bias or excess variance in the projected lightcone statistics, is discussed



Figure 2: [lesgourgues'massive'2006]

in detail by [kacprzak'cosmogridv1'2023], who additionally introduce a shell-permutation scheme that mitigates the repeated-structure problem by drawing different radial shells of a single lightcone from independent simulations of the same cosmology.

2.2 Cosmological Simulations

Cosmological N -body simulations have historically tracked, and in some respects driven, the development of high-performance computing: the number of particles that can be evolved in a fixed wall-clock time has grown by many orders of magnitude over the past decades, both because of raw hardware improvements and because of continual gains in the efficiency of the gravity solvers themselves [potter'pkdgrav3'2016]. The scientific payoff of these simulations lies in their ability to model the large-scale structure (LSS) of the Universe to the non-linear precision that photometric surveys require. LSS observables are particularly sensitive to the present-day matter density Ω_m and the amplitude of matter clustering σ_8 ; because a lightcone simulation can be sliced into tomographic redshift bins, it further allows the growth of structure to be tracked over cosmic time, which is what makes such simulations sensitive to the dark-energy equation-of-state parameter w_0 (and its possible time evolution) and hence to the physical origin of cosmic acceleration [kacprzak'cosmogridv1'2023].

2.2.1 DISCO-DJ

All forward models implemented in Disco-Dj are concerned with computing an approximate solution to the cosmological Vlasov–Poisson system:

$\frac{df}{dt} = \frac{df}{dt} + \mathbf{p} \cdot \frac{d\mathbf{x}}{dt} + \mathbf{g} \cdot \frac{d\mathbf{p}}{dt} = 0$, where $\mathbf{g} := \mathbf{x}$, (2.1) see e.g. [56, 57] for recent reviews. Here, $f = f(\mathbf{x}, \mathbf{p}, t)$ is the phase-space distribution function, where the coordinates $\mathbf{x}$ are comoving in a universe expanding with a scale factor $a(t)$.

We now turn towards the ingredients for PM (particle mesh) simulations. A key component is the time integration scheme used for the temporal discretisation of the Vlasov–Poisson system (2.1). To make a connection with the previous sections, we note that it is possible in fact to design integrators that exactly trace perturbative trajectories up to a certain order, while higher-order truncation errors decrease as the step size is reduced [78–80]. A single PM time step would then compute, e.g., 2LPT trajectories (up to higher-order terms, and with the caveat that discretisation errors arise due to the Eulerian PM-based force computation, compared to the Lagrangian evaluation of quantities in LPT). Disco-Dj supports different drift-kick-drift leapfrog/Verlet steppers of the form:

$\mathbf{X}_{n+1/2}^i = \mathbf{X}_n^i + \mathbf{V}_n^i \Delta t$, $\mathbf{V}_{n+1}^i = \mathbf{V}_n^i + \mathbf{A}(\mathbf{X}_{n+1/2}^i) \Delta t$, $\mathbf{X}_{n+1}^i = \mathbf{X}_{n+1/2}^i + \mathbf{V}_{n+1}^i \Delta t$. Here, $\mathbf{X}_n^i$ denotes the position of particle i at the n th time step, $\mathbf{A}(\mathbf{X}_{n+1/2}^i)$ is the acceleration evaluated in the middle of the full time step (i.e. after the first drift), $\Delta t = 1 + 2$ is the time step size, and α and β are coefficients that may depend on both the current time and time step. For the

velocity variable $V_{n,i}$ (and the associated time step variable Δt_n), different choices are supported, such as • the canonical momentum variable $V = a^2 \dot{X}$ with $\Delta t_n = R_{n+1} - R_n / (H(a) a^3)^{1/2}$ and • the growth-time velocity $V = dX/dD = X / (aH(a)D(a))$ with $\Delta t_n = D_{n+1} - D_n$, where the overdot denotes a derivative w.r.t. cosmic time. symplectic Zeldovich consistent integrator. Recently, Ref. [80] built onto the analysis of Ref. [79] and derived the BullFrog integrator, which corresponds to the unique choice of Δt_n and V that aligns the N -body trajectory with 2LPT after each time step in its regime of validity: $\Delta t_n = E(D_{n+1}) - E(D_n) / (D_{n+1} - D_n)^{1/2}$, $V = (1 - \epsilon) D_{n+1/2}$, (2.16) – 10 where $\epsilon = D_1 - D_{n+1/2} / E_n + E(D_n) / D_{n+1} - D_n$ (2.17) and $E = (3/7)D^2 - (3/1001)D^5 + O(D^8)$ with $:= \ddot{m}$ is the second-order growth function (see Appendix B for the defining ordinary differential equation). 1 In view of its superior performance, BullFrog is the default integrator in DiscoDJ, yielding accurate predictions already with very few steps. [list‘disco-dj’2026]

2.2.2 Pkdgrav

The gravity solver PKDGRAV3 computes the gravitational force with a binary-tree Fast Multipole Method (FMM): rather than only approximating a distant tree cell’s effect on individual particles (as in a Barnes–Hut tree code), it also expands the potential *within* a receiving cell sourced by a distant cell, an interaction that lets the cost of a full force evaluation scale as $\mathcal{O}(N)$ instead of $\mathcal{O}(N \log N)$ [potter‘pkdgrav3’2016].

Because particles in different regions of a cosmological box span many orders of magnitude in local density, and hence in dynamical time, a single global time step would be highly wasteful. PKDGRAV3 instead assigns each particle to one of several time-step “rungs”, advancing only the particles active on a given rung at each sub-step, and additionally builds a second, smaller tree restricted to the small subset of particles on the shortest (“very active”) rungs so that this frequently-rebuilt tree stays cheap; the combination is summarised by the kick-drift-kick “umbrella” diagram over one base time step, see Figure ?? . This dual-tree multi-stepping scheme is what keeps the tree-construction cost from dominating the runtime even though the shortest rungs may be walked hundreds of times per base step [potter‘pkdgrav3’2016].

2.2.3 CosmoGridV1

CosmoGridV1 is a large, publicly released suite of lightcone N -body simulations built specifically for map-level (rather than two-point-function-only) cosmological inference from large-scale-structure probes such as weak lensing and galaxy clustering [kacprzak‘cosmogridv1’2023]. It samples a six-dimensional flat w CDM parameter space $(\Omega_m, \sigma_8, w_0, H_0, n_s, \Omega_b)$, with a fixed total neutrino mass) at 2500 points arranged on a Sobol sequence, running 7 independent realisations at every point, in addition to 200 realisations (with parameter-derivative

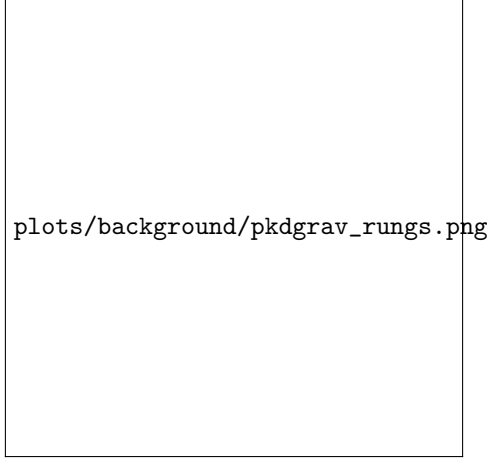


Figure 3: [potter'pkdgrav3'2016]

pairs) at a fixed fiducial cosmology and a smaller set of benchmark runs at higher resolution used to validate the main configuration's choices of box size, particle count, and shell thickness [kacprzak'cosmogridv1'2023]. Every simulation is run with PKDGRAV3 in a periodic box of $900 \text{ Mpc}/h$ with 832^3 particles, and stores its lightcone as HEALPix particle-count maps at $N_{\text{side}} = 2048$ out to $z = 3.5$, together with friends-of-friends halo catalogues at each output step [kacprzak'cosmogridv1'2023]. These raw particle-count shells are the basis from which observable maps of weak-lensing convergence, galaxy clustering, and intrinsic alignment are subsequently constructed, and the same dataset has previously been used, together with a shell-based baryon correction model, for a deep-learning cosmological parameter analysis of the KiDS-1000 weak-lensing survey [fluri'full'2022].

3 Methods and implementation

3.1 Pipeline Architecture

Limitations and Solutions

The problem of cosogrid and pkdgrav is that we have much longer simulation time. The alternative framework to it is by using the fast disco simulation. The tradeoff we have with disco here is that we have less precision on small scales. So to overall goal is:

Let $f(\text{Map})$ be some small scale corrections function

$$f(\text{Map}_{\text{disco}}) \approx \text{Map}_{\text{Cosmogrid}}$$

First we planned to do an AI-correction, but a better approach was with correcting the angular power matter spectrum for high dipole l .

3.2 IC generation

3.2.1 Validation against CosmoGridV1

Before using DISCO’s shell builder (Sec. ??) for any downstream analysis, we first validate that a lightcone produced with our PKDGRAV3 configuration reproduces the reference CosmoGridV1 statistics. To this end we run PKDGRAV3 with its native lightcone module on the exact box, resolution, and cosmology used by CosmoGridV1, and compare the resulting matter angular power spectrum C_ℓ against the corresponding CosmoGridV1 lightcone. Table ?? lists the parameters that must be matched exactly for this comparison to be meaningful, together with the corresponding keys in the PKDGRAV3 configuration file (`cosmology.par`).

3.2.2 Backscaled initial conditions and hand-off to disco

As described in section ?? we have to ensure that we are dealing with deliver a correct transfer function with the correct neutrino mass. Since this has an influence on directly, if it matches the `cosmogridv1`.

The resulting particle positions and velocities at $a_i = 1/(1+z_i)$ are exported and used, unmodified, as the initial condition for the corresponding DISCO run: DISCO evolves this state forward to $a = 1$ and accumulates the lightcone with the shell builder of Sec. ?? over the same redshift range, $z_{\max} = 3.5$ down to $z = 0$, at the same HEALPix resolution $N_{\text{side}} = 2048$.

3.3 Shell Builder

3.3.1 Core

A lightcone output requires the particle distribution to be resampled at every integration step, since the observable past lightcone shell changes continuously as the universe evolves. The most direct implementation stores the full particle snapshot at each of the n_{steps} integration steps and performs the shell decomposition after the simulation has finished. This approach is memory-prohibitive: retaining n_{steps} full-resolution snapshots simultaneously exceeds the available device memory for the particle counts considered here. We therefore interleave the shell decomposition with the time integration, so that at any given time only the particle states immediately before and after the current integration step, together with a set of low-dimensional pixelized count maps, need to be held in memory.

Time discretization. The radial shells are not defined by the integrator itself but are supplied externally, e.g. through a CosmoGridV1-format metainfo table. Each shell $s \in \{1, \dots, S\}$ is specified by a redshift interval $[z_s^{\text{lo}}, z_s^{\text{hi}}]$ and the corresponding comoving-distance interval $[r_s^{\text{lo}}, r_s^{\text{hi}}]$. To ensure that no shell boundary is crossed *within* an integration step, we define the set of integration

times as the union of all shell edges,

$$\{z_j\}_{j=0}^S = \text{unique}\left(\{z_s^{\text{lo}}\}_{s=1}^S \cup \{z_s^{\text{hi}}\}_{s=1}^S\right), \quad a_j = \frac{1}{1+z_j}, \quad (6)$$

sorted in ascending order of a_j (equivalently, descending order of z_j). By construction, every shell boundary coincides exactly with an integration-step boundary. The interval $[a_{\text{ini}}, a_0]$, preceding the first shell, is integrated separately with n_{pre} sub-steps and without shell accumulation, in order to bring the particle state into registry with the shell grid defined in Eq. (??). The remaining $J = S$ intervals $[a_j, a_{j+1}]$, $j = 0, \dots, J-1$, are each advanced with exactly one integration sub-step.

Accumulation procedure. Algorithm ?? summarizes the per-step accumulation. At step j , the comoving positions and validity mask of all particles are recorded both before the sub-step, $\{\mathbf{x}_p^{(j)}, m_p^{(j)}\}$, and after it, $\{\mathbf{x}_p^{(j+1)}, m_p^{(j+1)}\}$. The scale-factor edges of the step are converted to comoving distance through the cosmology-dependent map $\chi(a)$,

$$R_j = \chi(a_j), \quad R_{j+1} = \chi(a_{j+1}), \quad R_j > R_{j+1}, \quad (7)$$

where the inequality follows from χ being strictly decreasing on $a \in (0, 1]$. The set of shells that are traversed during step j is

$$\mathcal{A}_j = \{s : r_s^{\text{lo}} < R_j \wedge r_s^{\text{hi}} > R_{j+1}\}, \quad (8)$$

i.e. those shells whose radial bin overlaps the annulus $[R_{j+1}, R_j]$ swept out during the step. For each active shell $s \in \mathcal{A}_j$, the pre- and post-step particle states, together with the radial bounds $(r_s^{\text{lo}}, r_s^{\text{hi}})$, are passed to the shell-accumulation routine described in Sec. ??, which returns a HEALPix pixel count map $\mathbf{M}_s^{(j)} \in \mathbb{N}^{N_{\text{pix}}}$. This map is added to a running total,

$$\mathbf{S}_s \mathrel{+}= \mathbf{M}_s^{(j)}, \quad s \in \mathcal{A}_j. \quad (9)$$

Because the integration grid is constructed from the shell edges themselves (Eq. ??), $\mathcal{A}_j$ reduces to a single shell in the generic case; the overlap criterion in Eq. (??) is retained in its general form to also cover degenerate or adjacent bins in the input shell table.

Output. Once all J steps have been processed, the set of per-shell count maps $\{\mathbf{S}_s\}_{s=1}^S$, together with the shell table, is serialized to a single output file. At no point during the procedure are more than $\mathcal{O}(1)$ full particle snapshots and S integer-valued maps of size N_{pix} held simultaneously; this is what removes the memory bottleneck associated with the naive end-of-run decomposition.

3.3.2 Crossing Condition

The condition follows the moving-lightcone formulation of PKDGRAV3 [noauthor'examplepkdgrav3'lightco exactly, rather than the vector-trajectory intersection used in the earlier sketch.

Algorithm 1 Streaming shell accumulation

```

1: Load shell table  $\{(z_s^{\text{lo}}, z_s^{\text{hi}}, r_s^{\text{lo}}, r_s^{\text{hi}})\}_{s=1}^S$ ; build integration grid  $\{a_j\}_{j=0}^J$ 
   (Eq. ??)
2: Initialize  $\mathbf{S}_s \leftarrow \mathbf{0}$  for all  $s$ 
3: Integrate particle state from  $a_{\text{ini}}$  to  $a_0$  with  $n_{\text{pre}}$  sub-steps (no accumulation)
4: for  $j = 0, \dots, J - 1$  do
5:   Record pre-step positions and mask,  $\{\mathbf{x}_p^{(j)}, m_p^{(j)}\}$ 
6:   Integrate one sub-step:  $a_j \rightarrow a_{j+1}$ 
7:   Record post-step positions and mask,  $\{\mathbf{x}_p^{(j+1)}, m_p^{(j+1)}\}$ 
8:   Compute  $R_j, R_{j+1}$  (Eq. ??) and active shells  $\mathcal{A}_j$  (Eq. ??)
9:   for  $s \in \mathcal{A}_j$  do
10:     $\mathbf{M}_s^{(j)} \leftarrow$  crossing map from  $\{\mathbf{x}_p^{(j)}, \mathbf{x}_p^{(j+1)}, m_p^{(j)}, m_p^{(j+1)}\}$  restricted to
       $[r_s^{\text{lo}}, r_s^{\text{hi}}]$  (Sec. ??)
11:     $\mathbf{S}_s += \mathbf{M}_s^{(j)}$ 
12:   end for
13:   Discard pre-step state; retain post-step state as pre-step state for  $j + 1$ 
14: end for
15: return  $\{\mathbf{S}_s\}_{s=1}^S$ 

```

Periodic replicas. The observer is placed at the box centre, $\mathbf{o} = (L/2, L/2, L/2)$, with L the box side length. Since the requested lightcone depth $\chi_{\text{max}} = \chi(1/(1+z_{\text{max}}))$ generally exceeds L , the simulation volume is periodically replicated: let

$$\mathcal{D} = \left\{ \mathbf{d} = (n_1, n_2, n_3) L : \mathbf{n} \in \mathbb{Z}^3, \text{ replicated cube intersects the ball of radius } \chi_{\text{max}} \text{ about } \mathbf{o} \right\} \quad (10)$$

denote the finite set of replica offsets, precomputed once from (χ_{max}, L) . For each $\mathbf{d} \in \mathcal{D}$ we also precompute the nearest and farthest possible distance of the replicated cube from the observer,

$$r_{\text{min}}(\mathbf{d}) = \|\max(0, |\mathbf{d}| - L/2)\|, \quad r_{\text{max}}(\mathbf{d}) = \|\mathbf{d}| + L/2\|, \quad (11)$$

(componentwise max and absolute value, followed by the Euclidean norm), which allow a replica to be rejected for a given step without evaluating any particle if its radial range cannot overlap the step's lightcone annulus.

Linear crossing formula. For step j with N-body boundaries $a_j < a_{j+1}$, define, as before, $r_j^{\text{hi}} = \chi(a_j)$ and $r_j^{\text{lo}} = \chi(a_{j+1})$. For a particle with pre-/post-step positions $\mathbf{x}_p(a_j), \mathbf{x}_p(a_{j+1})$ relative to the box centre, and a replica offset $\mathbf{d} \in \mathcal{D}$, define the shifted positions and their radii

$$\mathbf{R}_0 = \mathbf{x}_p(a_j) + \mathbf{d}, \quad \mathbf{R}_1 = \mathbf{x}_p(a_{j+1}) + \mathbf{d}, \quad r_0 = \|\mathbf{R}_0\|, \quad r_1 = \|\mathbf{R}_1\|. \quad (12)$$

Rather than solving for the intersection of the interpolated *vector* trajectory with the receding lightcone sphere (which requires a quadratic root), the implementation follows PKDGRAV3 and linearly interpolates the *scalar* radii of the particle and of the lightcone front over the step,

$$r_p(v) = r_0 + v(r_1 - r_0), \quad R(v) = r_j^{\text{hi}} - v(r_j^{\text{hi}} - r_j^{\text{lo}}), \quad v \in [0, 1], \quad (13)$$

and solves $r_p(v^*) = R(v^*)$ for the crossing parameter,

$$v^* = \frac{r_j^{\text{hi}} - r_0}{(r_j^{\text{hi}} - r_j^{\text{lo}}) + (r_1 - r_0)}. \quad (14)$$

A particle–replica pair is accepted only if $v^* \in [0, 1]$; since Eq. (??) partitions the step interval, this guarantees that every particle is caught in *at most one* step, for a fixed replica – evaluating the crossing test against the (generally narrower) metainfo shell bounds instead of the full step bounds $[r_j^{\text{lo}}, r_j^{\text{hi}})$ would break this partition and could double-count particles moving across two adjacent shells within the same step. Pairs with $|r_1 - r_0|$ below a numerical tolerance (10^{-20}) are treated as non-moving and rejected, and the denominator of Eq. (??) is floored in magnitude at 10^{-10} to avoid division by a near-zero value.

Shell assignment and pixelization. The crossing position is obtained by applying v^* to the *vector* positions,

$$\mathbf{x}_{\text{cross}} = (1 - v^*) \mathbf{R}_0 + v^* \mathbf{R}_1, \quad r_{\text{cross}} = \|\mathbf{x}_{\text{cross}}\|, \quad (15)$$

and the pair is finally kept only if its crossing radius falls inside the metainfo (or, in the absence of an override, the step) shell bounds,

$$r_s^{\text{lo}} \leq r_{\text{cross}} < r_s^{\text{hi}}. \quad (16)$$

Surviving crossings are pixelized with the equal-area HEALPix ring scheme at resolution N_{side} from the unit vector $\hat{\mathbf{x}}_{\text{cross}} = \mathbf{x}_{\text{cross}}/r_{\text{cross}}$, and the corresponding pixel of the shell count map is incremented by one. Algorithm ?? summarizes the full test, which is applied to every particle and every replica $\mathbf{d} \in \mathcal{D}$ surviving the fast reject of Eq. (??), for every step j and every active shell $s \in \mathcal{A}_j$.

This procedure reproduces the corner-observer, box-replication scheme of PKDGRAV3 [noauthor:examplepkdgrav3:lightcone:2022] almost exactly, with two differences: the observer is placed at the box centre rather than a corner, and the replica set $\mathcal{D}$ (Eq. ??) is sized adaptively from the requested depth χ_{max} and box length L – together with a per-step, per-replica radial reject (Eq. ??) – rather than fixed a priori to $6^3 = 216$ images.

3.4 Neural Network

3.4.1 Dimensional Reduction

VRAM issues - $\hat{\mathbf{r}}_i$ patches or in spherical harmonics

Algorithm 2 Lightcone crossing test (per step j , per replica $\mathbf{d}$)

Require: positions $\mathbf{x}_p(a_j), \mathbf{x}_p(a_{j+1})$; step radii $r_j^{\text{lo}}, r_j^{\text{hi}}$; shell bounds $r_s^{\text{lo}}, r_s^{\text{hi}}$; replica $\mathbf{d}$

- 1: $\mathbf{R}_0 \leftarrow \mathbf{x}_p(a_j) + \mathbf{d}, \quad \mathbf{R}_1 \leftarrow \mathbf{x}_p(a_{j+1}) + \mathbf{d}$
- 2: $r_0 \leftarrow \|\mathbf{R}_0\|, \quad r_1 \leftarrow \|\mathbf{R}_1\|$
- 3: **if** $|r_1 - r_0| \leq \epsilon_{\text{move}}$ **then**
- 4: **return** *reject (stationary)*
- 5: **end if**
- 6: $v^* \leftarrow (r_j^{\text{hi}} - r_0) / [(r_j^{\text{hi}} - r_j^{\text{lo}}) + (r_1 - r_0)] \triangleright \text{Eq. ??, denominator floored at } \epsilon_{\text{den}}$
- 7: **if** $v^* \notin [0, 1)$ **then**
- 8: **return** *reject (not caught this step)*
- 9: **end if**
- 10: $\mathbf{x}_{\text{cross}} \leftarrow (1 - v^*)\mathbf{R}_0 + v^*\mathbf{R}_1, \quad r_{\text{cross}} \leftarrow \|\mathbf{x}_{\text{cross}}\|$
- 11: **if** $r_{\text{cross}} \notin [r_s^{\text{lo}}, r_s^{\text{hi}})$ **then**
- 12: **return** *reject (wrong shell)*
- 13: **end if**
- 14: $\text{pix} \leftarrow \text{VEC2PIX}(N_{\text{side}}, \hat{\mathbf{x}}_{\text{cross}})$
- 15: $\mathbf{S}_s[\text{pix}] += 1$

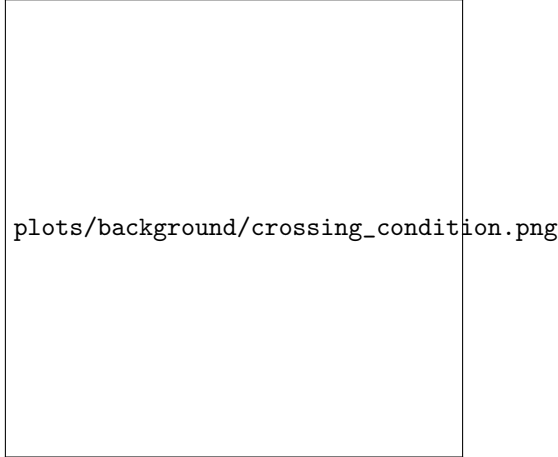


Figure 4: [noauthor·examplepkdgrav3·lightcone·2022]

3.4.2 Data preparation

3.4.3 Network Architecture

3.4.4 Training Procedure

3.4.5 Validation Strategy

(Angular) Power Spectru, Kappa -i UFALCON

4 Computational Infrastructure

4.1 Computing Environments

The code development went through several stages. Starting locally with only testing and seeing how things works towards more complex architecture inside of an cluster.

4.1.1 Local Development

We first implemented disco and pkdgrav on a local device, to see what it needs to run. This also makes sense, since one have direct ouptut, which can be checked and fixed directly. This much more time efficient, than when dealing with clusters. For some simple tests we used a low resolution $res = 16$, which lead to a total particle number of $16^3 = 4096$. A local setup can easily deal with it and also terminate under minute. The problem starts when we want to have higher resolutions, this lead directly to RAM error, since

$$res = 832 \rightarrow 832^3 = 5 \cdot 10^8 \text{ particles} \rightarrow \dots \text{GB},$$

which is mostly not available on simple computers.

4.1.2 Euler (ETH)

On the cluster we have created virtual environments for each module (pkdgrav, disco), since some libraries conflict with others and therefore we have to treat it seperately. But regarding the resources we are now able to use higher resolutions as $res = 832$. We can also use GPU supported version, which can lead to a massive speed up. Nevertheless the allocations for this simulations can lead to a rather long runtime depending on what configuration is used. We can therefore adapt to handle multi-node configurations.

4.1.3 CSCS (Alps/Piz Daint)

Here we can use the very efficient NVIDIA GH200 with 120 GB of VRAM. This is perfect for scaling up and to reduce the runtimes for each simulation, either for generating the training pairs or for a training of the model itself.

4.2 Software Engineering

4.2.1 Code Structure

We saved all the important executable and virtual environments side of the HOME directory of the cluster, since it's not dropped at any times. Since the storage in home is limited to a specific amount, we saved the output into SCRATCH. For example we submitted a transfer job from globus to endpoint of CSCS to transfer all data from Cosmogridv1 of size 110 *TB*. But also for sure much smaller size of data as for example all logs for each job submission.

This was very helpful for debugging and also for later look ups, to compare. Also important plots of snapshots and shells are stored inside of that.

4.2.2 Job Scheduling and Submissions

For each python script we have also created an bash file. This was very useful since the python script could be used locally, but also on the cluster and the bash file of it was inside of the cluster for job submission either with *sbatch* or *bash*. That way we were able to control, if we want to monitor output very fast inside of the terminal or if we want to run it in the background and store it.

4.2.3 Version Control (GitHub)

A very important decision was to create a new repository for the project (all scripts) and also the modified DISCO with shell builder. This is useful to have certain safety, if the computer crashes or when losing overview. So there is always the possibility to restore an older working version.

5 Results

from [defferrard'deepisphere'2020]

6 Conclusion